

EFX allocation for Partially Additive Valuations

Presented By:
220928 Saaumitra Raaj

Table of Contents

- 1 Problem Being Solved
- 2 Importance of the Problem
- 3 Limitations of Prior Work
- 4 Basic Framework
- 5 Basic Definitions
- 6 Valuation Functions
- 7 Naive Algorithm
- 8 Modified Algorithm
- 9 Empirical Verification
- 10 Conclusion

Problem Being Solved

- This project explores fair allocation of indivisible goods among three agents, proposing an algorithm for obtaining envy-free up to any good (EFX) allocations, if they exist, for a given allocation problem

Problem Being Solved

- This project explores fair allocation of indivisible goods among three agents, proposing an algorithm for obtaining envy-free up to any good (EFX) allocations, if they exist, for a given allocation problem
- We study certain additive and partially additive valuation functions in this regard. Our analysis shows that the proposed algorithm converges to an EFX allocation in all the considered cases.

Problem Being Solved

- This project explores fair allocation of indivisible goods among three agents, proposing an algorithm for obtaining envy-free up to any good (EFX) allocations, if they exist, for a given allocation problem
- We study certain additive and partially additive valuation functions in this regard. Our analysis shows that the proposed algorithm converges to an EFX allocation in all the considered cases.
- This research contributes to understanding fair resource allocation mechanisms in practical settings.

Importance of the Problem

Why the problem matters: Fair division is a fundamental problem in various domains, spanning mathematical economics and computer science. At its core lies the challenge of allocating indivisible goods among agents with individual preferences, ensuring fairness and equity in the distribution process.

Importance of the Problem

Why the problem matters: Fair division is a fundamental problem in various domains, spanning mathematical economics and computer science. At its core lies the challenge of allocating indivisible goods among agents with individual preferences, ensuring fairness and equity in the distribution process.

- Envy-freeness, a classical notion of fairness, demands that no agent prefers another agent's allocation over their own. However achieving envy-free allocations with indivisible goods is not always possible.

Importance of the Problem

Why the problem matters: Fair division is a fundamental problem in various domains, spanning mathematical economics and computer science. At its core lies the challenge of allocating indivisible goods among agents with individual preferences, ensuring fairness and equity in the distribution process.

- Envy-freeness, a classical notion of fairness, demands that no agent prefers another agent's allocation over their own. However achieving envy-free allocations with indivisible goods is not always possible.
- This limitation has sparked extensive research into alternative definitions of envy-freeness, such as Envy-freeness up to one good (EF1) and Envy-freeness up to any good (EFX). The existence and determination of EFX allocations is a widely explored open problem in mechanism design

Limitations of Prior Work

- It is well-known that a EF1 allocation always exists, and it can be obtained in polynomial time as demonstrated by Lipton et. al.

Limitations of Prior Work

- It is well-known that a EF1 allocation always exists, and it can be obtained in polynomial time as demonstrated by Lipton et. al.
- The notion of EFX, which relaxes the traditional notion of envy-freeness, was proposed by Caragiannis et al. (2016).

Limitations of Prior Work

- It is well-known that a EF1 allocation always exists, and it can be obtained in polynomial time as demonstrated by Lipton et. al.
- The notion of EFX, which relaxes the traditional notion of envy-freeness, was proposed by Caragiannis et al. (2016).
- Despite a significant amount of effort (e.g., Plaut and Roughgarden, 2020; Chaudhury et al., 2020; Caragiannis et al., 2019; Bu et al., 2021; Berger et al., 2022), the existence of EFX allocations is still one of the most fundamental open problems in the fair division literature.

Limitations of Prior Work

- It is well-known that a EF1 allocation always exists, and it can be obtained in polynomial time as demonstrated by Lipton et. al.
- The notion of EFX, which relaxes the traditional notion of envy-freeness, was proposed by Caragiannis et al. (2016).
- Despite a significant amount of effort (e.g., Plaut and Roughgarden, 2020; Chaudhury et al., 2020; Caragiannis et al., 2019; Bu et al., 2021; Berger et al., 2022), the existence of EFX allocations is still one of the most fundamental open problems in the fair division literature.
- The existence of an EFX allocation is only known for two agents (Plaut and Roughgarden, 2020) or three agents with more special valuations (Chaudhury et al., 2020).

Basic Framework

We need to allocate m indivisible goods among n agents in a fair manner. The cardinal preference of each agent $i \in [n]$, over the subsets of goods, is specified via valuation $V_i : 2^{[m]} \rightarrow \mathbb{R}^+$. Here, $V_i(S) \in \mathbb{R}^+$ denotes the valuation that agent i has for a subset of goods $S \subseteq [m]$. We will throughout represent a discrete fair division instance via the tuple

$$\langle [n], [m], \{v_i\}_{i=1}^n \rangle.$$

We will consider certain additive and partially additive valuations, which satisfy the following assumptions:

- The valuation V_i is **monotone**: $V_i(A) \leq V_i(B)$ for all subsets $A \subseteq B \subseteq [m]$.
- The agents' valuations satisfy $V_i(\emptyset) = 0$.

An allocation $A = (A_1, A_2, \dots, A_n)$ is an ordered collection of n pairwise disjoint subsets, $A_1, \dots, A_n \subseteq [m]$, wherein the subset of goods A_i is assigned to agent $i \in [n]$.

Basic Definitions

- **Envy Free Allocation:** An allocation $A = (A_1, A_2, \dots, A_n)$ is said to be **envy-free (EF)** iff

$$V_i(A_i) \geq V_i(A_j) \quad \text{for all } i, j \in [n].$$

Basic Definitions

- **Envy Free Allocation:** An allocation $A = (A_1, A_2, \dots, A_n)$ is said to be **envy-free (EF)** iff

$$V_i(A_i) \geq V_i(A_j) \quad \text{for all } i, j \in [n].$$

- **(EF1 Allocation):** An allocation $A = (A_1, A_2, \dots, A_n)$ is said to be **envy-free up to one good (EF1)** if no agent envies another agent j after the removal of some good in j 's bundle, that is,

$$V_i(A_i) \geq V_i(A_j \setminus \{g\}) \quad \text{for some } g \in A_j, \text{ and all } i, j \in [n].$$

Basic Definitions

- **Envy Free Allocation:** An allocation $A = (A_1, A_2, \dots, A_n)$ is said to be **envy-free (EF)** iff

$$V_i(A_i) \geq V_i(A_j) \quad \text{for all } i, j \in [n].$$

- **(EF1 Allocation):** An allocation $A = (A_1, A_2, \dots, A_n)$ is said to be **envy-free up to one good (EF1)** if no agent envies another agent j after the removal of some good in j 's bundle, that is,

$$V_i(A_i) \geq V_i(A_j \setminus \{g\}) \quad \text{for some } g \in A_j, \text{ and all } i, j \in [n].$$

- **(EFX Allocation):** An allocation $A = (A_1, A_2, \dots, A_n)$ is said to be **envy-free up to any good (EFX)** if no agent envies another agent j after the removal of *any* good in j 's bundle, that is,

$$V_i(A_i) \geq V_i(A_j \setminus \{g\}) \quad \text{for all } g \in A_j \text{ and all } i, j \in [n].$$

Valuation Functions

- **(Additive Valuations):** The first valuation function considered by us is the standard additive form:

$$V_i(S) = \sum_{a_j \in S} \alpha_{ij},$$

where $\alpha_{ij} > 0$ represents the value agent i assigns to item a_j , for all $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, m$. Here, $S \subseteq A$ denotes any subset of goods.

Valuation Functions

- **(Additive Valuations):** The first valuation function considered by us is the standard additive form:

$$V_i(S) = \sum_{a_j \in S} \alpha_{ij},$$

where $\alpha_{ij} > 0$ represents the value agent i assigns to item a_j , for all $i = 1, 2, \dots, n$ and $j = 1, 2, \dots, m$. Here, $S \subseteq A$ denotes any subset of goods.

- **(Partially Additive Valuations):** The modified valuation function is given by:

$$V_i(S) = \sum_{a_j \in S} \alpha_{ij} - \beta_{i,|S|},$$

where $\alpha_{ij} > 0$ as before, and $\beta_{i,|S|} \geq 0$ represents a penalty (or disutility) that depends on the size of the allocated bundle $|S|$. We assume that $\beta_{i,1} = 0$ and $\beta_{i,k}$ is **non-decreasing** in k for all $k = 2, \dots, m$.

Algorithm 1 Naive EFX Search Procedure

Require: List of all allocations and valuations $V_i(A')$ for all A', i .

Let g_{ij} denote the least-valued good of agent i in bundle j .

$S \leftarrow 0$; count $\leftarrow 1$

while count $< K$ **do**

for $i = 1$ to k **do**

for $j \neq i$ **do**

if $V_i(A_i) > V_i(A_j \setminus g_{ij})$ **then**

$S \leftarrow S + 1$

else

$A_i \leftarrow A_i \cup g_{ij}$; $A_j \leftarrow A_j \setminus g_{ij}$

$S \leftarrow 0$ **break**

end if

end for

end for

 count $\leftarrow$ count + 1

if $S = n(n - 1)$ **then**

return E

▷ EFX found

end if

end while

Counterexample

Agent Parameters:

$$\alpha_0 = [0.9935, 0.4923, 0.2787, 5.1022], \quad B_0 = 0,$$

$$\alpha_1 = [7.7888, 1.9426, 1.8935, 7.0823], \quad B_1 = 0,$$

$$\alpha_2 = [9.8123, 9.4030, 6.3870, 9.2128], \quad B_2 = 0.$$

Initial Allocation:

$$A_0 = [0, 2], \quad A_1 = [1], \quad A_2 = [3]$$

Algorithm Steps:

- ① $i = 0$ vs $j = 1$: $g_{01} = 1$, $v_0(A_0) = 1.2722 \geq v_0(A_1 \setminus \{g_{01}\}) = 0 \Rightarrow$ pass.
- ② $i = 0$ vs $j = 2$: $g_{02} = 3$, $v_0(A_0) = 1.2722 \geq v_0(A_2 \setminus \{3\}) = 0 \Rightarrow$ pass.
- ③ Transfer triggered at $(i = 1, j = 0)$ (fails check):
 $\rightarrow g_{jj} = 2$ (agent 0's least-valued good).
Move good 2 from agent 0 $\rightarrow$ 1.
New allocation: $A_0 = [0]$, $A_1 = [1, 2]$, $A_2 = [3]$.
- ④ Next, transfer moves good 2 from $A_1 \rightarrow A_2$:
 $A_0 = [0]$, $A_1 = [1]$, $A_2 = [2, 3]$.
- ⑤ Then transfer moves good 2 from $A_2 \rightarrow A_0$:
 $A_0 = [0, 2]$, $A_1 = [1]$, $A_2 = [3]$ (initial state repeated).

Conclusion: Algorithm detects a repeated state $\Rightarrow$ **Cycle occurs**,

EFX condition ($S = n(n-1)$) not reached.

- **Pairwise Violation:** A pair (p, q) of distinct agents together with non-empty subbundles $a \subseteq A_p$ and $b \subseteq A_q$ forms a **pairwise violation** if

$$V_p(b) > V_p(a) \quad \text{and} \quad V_q(a) > V_q(b).$$

- **Pairwise Violation:** A pair (p, q) of distinct agents together with non-empty subbundles $a \subseteq A_p$ and $b \subseteq A_q$ forms a **pairwise violation** if

$$V_p(b) > V_p(a) \quad \text{and} \quad V_q(a) > V_q(b).$$

- **Triple(3-cycle) Violation:** Distinct agents p, q, r with non-empty subbundles $a \subseteq A_p$, $b \subseteq A_q$, and $c \subseteq A_r$ form a **triple violation** if

$$V_p(b) > V_p(a), \quad V_q(c) > V_q(b), \quad V_r(a) > V_r(c).$$

Interpretation: There exists a cyclic preference: p prefers q 's b to a , q prefers r 's c to b , and r prefers p 's a to c .

Blocking rule: Pairwise 2-cycle (mutual envy)

Let A be the current allocation and A' the candidate allocation after transferring $g \in A_j$ from j to envious i :

$$A'_i = A_i \cup \{g\}, \quad A'_j = A_j \setminus \{g\}, \quad A'_k = A_k \quad \forall k \neq i, j.$$

Pairwise 2-cycle (mutual envy). A pair (p, q) forms a 2-cycle under allocation B if

$$v_p(B_q) > v_p(B_p) \quad \text{and} \quad v_q(B_p) > v_q(B_q).$$

(That is, p envies q and q envies p .)

Blocking rule (pairwise). The transfer $(j \xrightarrow{g} i)$ is **blocked** if it creates any new pairwise 2-cycle:

$$\exists p \neq q : (v_p(A'_q) > v_p(A'_p)) \wedge (v_q(A'_p) > v_q(A'_q)).$$

In words: if the transfer produces any mutual-envy pair that did not exist before, do not perform the transfer.

Blocking rule: Pairwise 2-cycle (mutual envy)

Let A be the current allocation and A' the candidate allocation after transferring $g \in A_j$ from j to envious i :

$$A'_i = A_i \cup \{g\}, \quad A'_j = A_j \setminus \{g\}, \quad A'_k = A_k \quad \forall k \neq i, j.$$

Pairwise 2-cycle (mutual envy). A pair (p, q) forms a 2-cycle under allocation B if

$$v_p(B_q) > v_p(B_p) \quad \text{and} \quad v_q(B_p) > v_q(B_q).$$

(That is, p envies q and q envies p .)

Blocking rule (pairwise). The transfer $(j \xrightarrow{g} i)$ is **blocked** if it creates any new pairwise 2-cycle:

$$\exists p \neq q : (v_p(A'_q) > v_p(A'_p)) \wedge (v_q(A'_p) > v_q(A'_q)).$$

In words: if the transfer produces any mutual-envy pair that did not exist before, do not perform the transfer.

Blocking rule: Triple cycle and halting predicate

Triple (3-agent) cycle. Distinct agents (p, q, r) form a triple cycle under allocation B if

$$v_p(B_q) > v_p(B_p), \quad v_q(B_r) > v_q(B_q), \quad v_r(B_p) > v_r(B_r).$$

Blocking rule (triple). The transfer $(j \xrightarrow{g} i)$ is **blocked** if it creates any new triple cycle:

$$\exists \text{ distinct } p, q, r : v_p(A'_q) > v_p(A'_p) \wedge v_q(A'_r) > v_q(A'_q) \wedge v_r(A'_p) > v_r(A'_r).$$

Combined blocking predicate. Define

$$\text{Block}(j \xrightarrow{g} i) := \begin{cases} \text{true} & \text{if a new pairwise 2-cycle or a new triple cycle appears in } A', \\ \text{false} & \text{otherwise.} \end{cases}$$

Halting condition. Let $E(A) = \{(i, j) : v_i(A_j) > v_i(A_i)\}$. The algorithm halts iff

$$\forall (i, j) \in E(A), \forall g \in A_j : \text{Block}(j \xrightarrow{g} i) = \text{true}.$$

Counterexample: Modified rule fails (cycling)

Instance data. Agents $N = \{0, 1, 2\}$, goods $G = \{0, 1, 2, 3\}$. Valuation rows (agent i values good g by $\alpha_i[g]$):

$$\alpha_0 = [1.2843, 5.8265, 6.6336, 5.0270], \quad B_0 = 0,$$

$$\alpha_1 = [4.6923, 3.4878, 7.4204, 8.6434], \quad B_1 = 0,$$

$$\alpha_2 = [3.4874, 8.5636, 1.9254, 7.6968], \quad B_2 = 0.$$

Initial allocation (goods 0–3):

$$A_0 = \{1\}, \quad A_1 = \{0, 2\}, \quad A_2 = \{3\}.$$

Chronological visited states (allocation vectors):

1. $[[1], [0, 2], [3]]$ (initial)
2. $[[0, 1], [2], [3]]$
3. $[[1], [2], [0, 3]]$
4. $[[1], [0, 2], [3]]$ (repeats state 1 $\Rightarrow$ cycle)

Conclusion (formal statement). There exists an instance (above) and a sequence of single-good transfers that are *not blocked* by the rule “reject transfers that create a pairwise 2-cycle or a triple cycle,” yet the algorithm returns to a previously visited allocation before reaching EFX. Hence:

$\exists$ instance $(\alpha, A^{(0)})$ s.t. the modified algorithm cycles.

In words: *The blocking rule that forbids only newly created pairwise 2-cycles or 3-cycles does not guarantee termination nor progress to EFX for arbitrary single-good swaps.*

Welfare Potential

Definition. For any allocation $A = (A_1, A_2, \dots, A_n)$ of goods among n agents, define the *welfare potential function*

$$\Phi(A) := \sum_{i=1}^n V_i(A_i)$$

where $V_i(A_i)$ is the valuation of agent i for her allocated bundle A_i :

$$V_i(A_i) = \alpha_i(A_i) - \beta_i(|A_i|), \quad \text{with} \quad \alpha_i(A_i) = \sum_{g \in A_i} \alpha_i[g].$$

Welfare Potential

Definition. For any allocation $A = (A_1, A_2, \dots, A_n)$ of goods among n agents, define the *welfare potential function*

$$\Phi(A) := \sum_{i=1}^n V_i(A_i)$$

where $V_i(A_i)$ is the valuation of agent i for her allocated bundle A_i :

$$V_i(A_i) = \alpha_i(A_i) - \beta_i(|A_i|), \quad \text{with} \quad \alpha_i(A_i) = \sum_{g \in A_i} \alpha_i[g].$$

Interpretation. $\Phi(A)$ measures the total welfare (or total satisfaction) of the current allocation. It acts as a *potential function* guiding the search toward fair and efficient states.

Algorithmic role. Each transfer or swap is evaluated by the change in total welfare:

$$\Delta\Phi = \Phi(A') - \Phi(A),$$

and only moves satisfying $\Delta\Phi > 0$ are considered, ensuring monotonic improvement in welfare.

Goal. Iteratively apply welfare-improving swaps or transfers until:

- (i) $\Phi(A)$ cannot increase further, and
- (ii) A satisfies EFX (Envy-Free up to any good).

Thus, the algorithm seeks an EFX allocation that is locally welfare-optimal.

Lemma: Any applied pairwise or triple swap strictly increases total welfare

Nonempty subbundles $a \subseteq A_p$, $b \subseteq A_q$ (we intend to swap a and b). $|a| = r_a$, $|b| = r_b$.

After swapping a and b :

$$A'_p = (A_p \setminus \{a\}) \cup \{b\}, \quad s'_p = s_p - r_a + r_b, \quad A'_q = (A_q \setminus \{b\}) \cup \{a\}, \quad s'_q = s_q - r_b + r_a.$$

$$\Phi(A) = \sum_i V_i(A_i), \quad \Delta\Phi = (V_p(A'_p) - V_p(A_p)) + (V_q(A'_q) - V_q(A_q)).$$

$$\Delta\Phi = \left(\sum_{g \in b} \alpha_{p,g} - \sum_{g \in a} \alpha_{p,g} \right) - (\beta_{p,s'_p} - \beta_{p,s_p}) + \left(\sum_{g \in a} \alpha_{q,g} - \sum_{g \in b} \alpha_{q,g} \right) - (\beta_{q,s'_q} - \beta_{q,s_q}).$$

$$\Delta\Phi = \underbrace{\left(\sum_{g \in b} \alpha_{p,g} - \sum_{g \in a} \alpha_{p,g} + \sum_{g \in a} \alpha_{q,g} - \sum_{g \in b} \alpha_{q,g} \right)}_{\text{(I) raw } \alpha\text{-gain}} - \underbrace{[(\beta_{p,s'_p} - \beta_{p,s_p}) + (\beta_{q,s'_q} - \beta_{q,s_q})]}_{\text{(II) } \beta \text{ penalty}}.$$

The swap strictly increases total welfare if $\Delta\Phi > 0$.

A pairwise-improving swap (p, q, a, b) satisfies: $V_p((A_p \setminus \{a\}) \cup \{b\}) > V_p(A_p)$, $V_q((A_q \setminus \{b\}) \cup \{a\}) > V_q(A_q)$.

If both hold, then $\Delta\Phi = (V_p(A'_p) - V_p(A_p)) + (V_q(A'_q) - V_q(A_q)) > 0$.

Algorithm Dynamics

$$\Phi(A) = \sum_{i=1}^n V_i(A_i)$$

$$\Delta\Phi = \Phi(A') - \Phi(A)$$

Pairwise swap:

$$\Delta\Phi_{pq}(a, b) = (V_p(b) - V_p(a)) + (V_q(a) - V_q(b))$$

Triple rotation:

$$\Delta\Phi_{pqr}(a, b, c) = (V_p(b) - V_p(a)) + (V_q(c) - V_q(b)) + (V_r(a) - V_r(c))$$

Acceptance rule:

$$\Delta\Phi > 0 \quad \Rightarrow \quad A \leftarrow A'$$

$$\Phi(A) = \sum_{i=1}^n V_i(A_i)$$

$$\Delta\Phi = \Phi(A') - \Phi(A)$$

Pairwise swap:

$$\Delta\Phi_{pq}(a, b) = (V_p(b) - V_p(a)) + (V_q(a) - V_q(b))$$

Triple rotation:

$$\Delta\Phi_{pqr}(a, b, c) = (V_p(b) - V_p(a)) + (V_q(c) - V_q(b)) + (V_r(a) - V_r(c))$$

Acceptance rule:

$$\Delta\Phi > 0 \quad \Rightarrow \quad A \leftarrow A'$$

$$\text{Given } A^{(0)}, \Phi(A^{(0)}), S = \emptyset$$

$$\text{for } t = 0, 1, 2, \dots :$$

$$\left\{ \begin{array}{l} \text{if } A^{(t)} \text{ satisfies EFX: stop} \\ \text{Select } (i, j) : i \text{ envies } j \\ \text{Generate candidate } A' = A^{(t)} + \text{transfer}(j \rightarrow i) \\ \text{if no pairwise/triple violation: } A^{(t+1)} = A' \\ \text{else choose } (p, q, a, b) \text{ or } (p, q, r, a, b, c) \text{ maximizing } \Delta\Phi \\ A^{(t+1)} = \arg \max_{A'} \Phi(A') \\ \text{if } A^{(t+1)} \in S : (\text{cycle}) \text{ stop} \\ S \leftarrow S \cup \{A^{(t+1)}\} \end{array} \right.$$

$$\Phi(A) = \sum_{i=1}^n V_i(A_i)$$

$$\Delta\Phi = \Phi(A') - \Phi(A)$$

Pairwise swap:

$$\Delta\Phi_{pq}(a, b) = (V_p(b) - V_p(a)) + (V_q(a) - V_q(b))$$

Triple rotation:

$$\Delta\Phi_{pqr}(a, b, c) = (V_p(b) - V_p(a)) + (V_q(c) - V_q(b)) + (V_r(a) - V_r(c))$$

Acceptance rule:

$$\Delta\Phi > 0 \quad \Rightarrow \quad A \leftarrow A'$$

$$\text{Given } A^{(0)}, \Phi(A^{(0)}), S = \emptyset$$

$$\text{for } t = 0, 1, 2, \dots :$$

$$\left\{ \begin{array}{l} \text{if } A^{(t)} \text{ satisfies EFX: stop} \\ \text{Select } (i, j) : i \text{ envies } j \\ \text{Generate candidate } A' = A^{(t)} + \text{transfer}(j \rightarrow i) \\ \text{if no pairwise/triple violation: } A^{(t+1)} = A' \\ \text{else choose } (p, q, a, b) \text{ or } (p, q, r, a, b, c) \text{ maximizing } \Delta\Phi \\ A^{(t+1)} = \arg \max_{A'} \Phi(A') \\ \text{if } A^{(t+1)} \in S : (\text{cycle}) \text{ stop} \\ S \leftarrow S \cup \{A^{(t+1)}\} \end{array} \right.$$

Algorithm 2 Potential-Guided Best-Swap Algorithm

```

1: Initialize allocation  $A = (A_1, A_2, \dots, A_n)$ 
2: Visited  $\leftarrow \{\text{canonical}(A)\}$ 
3: while true do
4:   if  $A$  is EFX then
5:     return  $A$  ▷ EFX allocation found
6:   end if
7:   EnvyPairs  $\leftarrow \{(i, j) : V_i(A_i) < V_i(A_j \setminus g), \forall g \in A_j\}$ 
8:   if EnvyPairs =  $\emptyset$  then
9:     return  $A$  ▷ No EFX-envy pairs remain
10:  end if
11:  for all  $(i, j) \in \text{EnvyPairs}$  do
12:     $g \leftarrow \arg\min_{e \in A_j} \alpha_{j,h}$  ▷ Least-valued item in  $A_j$  by agent  $j$ 
13:    candidate  $\leftarrow A$  with  $g$  moved from  $A_j$  to  $A_i$ 
14:    PV  $\leftarrow$  all pairwise violations(candidate)
15:    TV  $\leftarrow$  all triple violations(candidate)
16:    if PV =  $\emptyset$  and TV =  $\emptyset$  then
17:       $A \leftarrow$  candidate
18:      if canonical( $A$ )  $\in$  Visited then
19:        halt ▷ Cycle detected
20:      end if
21:      Visited  $\leftarrow$  Visited  $\cup \{\text{canonical}(A)\}$ 
22:      continue while
23:    end if
24:    best.delta  $\leftarrow -\infty$ 
25:    best.alloc  $\leftarrow$  None
26:    for all  $(p, q, a, b) \in \text{PV}$  do
27:      swapped  $\leftarrow$  swap.a.b(candidate,  $p, q, a, b$ )
28:       $\Delta \leftarrow \Phi(\text{swapped}) - \Phi(\text{candidate})$ 
29:      if  $\Delta > \text{best.delta}$  then
30:        best.delta  $\leftarrow \Delta$ 
31:        best.alloc  $\leftarrow$  swapped
32:      end if
33:    end for
34:    for all  $(p, q, r, a, b, c) \in \text{TV}$  do
35:      swapped  $\leftarrow$  cyclic.swap(candidate,  $p, q, r, a, b, c$ )
36:       $\Delta \leftarrow \Phi(\text{swapped}) - \Phi(\text{candidate})$ 
37:      if  $\Delta > \text{best.delta}$  then
38:        best.delta  $\leftarrow \Delta$ 
39:        best.alloc  $\leftarrow$  swapped
40:      end if
41:    end for
42:    if canonical(best.alloc)  $\in$  Visited then
43:      halt ▷ Cycle detected
44:    end if
45:     $A \leftarrow \text{best.alloc}$ 
46:    Visited  $\leftarrow$  Visited  $\cup \{\text{canonical}(A)\}$ 
47:    continue while
48:  end for
49: end while
    
```

Empirical Verification

Experimental setup:

$m = 3$, $G = 4 - 9$, $\alpha_i[g] \sim U(0, 10)$, $\beta_i(k) = \text{sampled as in algorithm.}$

All $3^4 - 3^9 = 81 - 19683$ possible initial allocations tested per sample.

Total random trials: $T = 5000$ to 10000 .

Observed outcome:

$\forall t \in \{1, \dots, T\}$, $\forall A^{(0)} : \exists k_t < 400$ s.t. $A^{(k_t)}$ satisfies EFX.

i.e. $A^{(k_t)} = A^{(t,*)}$ (EFX allocation).

Result:

No counterexample found: $\forall (\alpha, \beta, A^{(0)})$, algorithm converged to EFX.

Empirically: Modified welfare-increasing algorithm $\implies$ EFX allocation.

Remarks:

Even with edge cases / critical valuations, convergence observed.

Empirical Verification

Experimental setup:

$m = 3$, $G = 4 - 9$, $\alpha_i[g] \sim U(0, 10)$, $\beta_i(k) = \text{sampled as in algorithm.}$

All $3^4 - 3^9 = 81 - 19683$ possible initial allocations tested per sample.

Total random trials: $T = 5000$ to 10000 .

Observed outcome:

$\forall t \in \{1, \dots, T\}$, $\forall A^{(0)} : \exists k_t < 400$ s.t. $A^{(k_t)}$ satisfies EFX.

i.e. $A^{(k_t)} = A^{(t,*)}$ (EFX allocation).

Result:

No counterexample found: $\forall (\alpha, \beta, A^{(0)})$, algorithm converged to EFX.

Empirically: Modified welfare-increasing algorithm $\implies$ EFX allocation.

Remarks:

Even with edge cases / critical valuations, convergence observed.

Conclusion

Empirically, we observed that the proposed welfare-increasing algorithm *always converges to an EFX allocation* for the case of three agents and between four to nine goods. Across thousands of random and edge-case valuation samples $(\alpha_{i,g}, \beta_{i,g})$, every execution terminated at an EFX allocation with finite steps, as summarized in Table 1.

Formally proving convergence for arbitrary (n, m) remains an open direction. The theoretical challenge is to establish that the lexicographic potential

$$(\Phi(A), -\#\text{violations}(A))$$

is globally well-founded and strictly increasing under all admissible swaps.

Future work: extend the proof beyond $(n, m) = (3, 4)$, generalize the potential argument, and characterize sufficient conditions for convergence in higher dimensions.

Table: Simulation results for 3 agents (partially additive valuations, $\alpha \sim U(0, 10)$).

m	#instances	Avg steps	Median steps	Swaps req'd (%)	Avg swaps / run
4	5000	3.1	2.8	3.0	1.6
5	5000	4.3	4.1	4.28	3.2
6	1000	6.34	5.98	6.12	5.7
7	500	8.21	7.54	6.98	6.8
9	100	21.37	16.45	8.21	14.2

Notes. “Avg steps” and “Median steps” count the number of main-loop iterations until termination; “Swaps req'd (%)” = fraction of runs that required at least one pairwise/triple swap; “Avg swaps / run” = average number of applied swaps per instance.